

Aufgabe 2: Simultane Labyrinth

Teilnahme-ID: 76626

Bearbeiter/-in dieser Aufgabe:
Quirin Klemm

13. April 2025

Inhaltsverzeichnis

1	Lösungsidee	2
1.1	Erstellung der Matrix	2
1.2	Lösen der Labyrinth	2
1.3	Gruben	3
2	Umsetzung	4
2.1	Erstellung der Matrix	4
2.2	Lösen der Labyrinth	5
3	Laufzeit und Diskussion	7
4	Beispiele	7
5	Quellcode	7

1 Lösungsidee

Gesucht ist möglichst kurze Abfolge von Schritten, die beide Labyrinth löst. Dafür wird zunächst für jedes Labyrinth eine Matrix erstellt (Abbildung 1), in der für jede Position ein Kostenwert und eine Richtung gespeichert wird. Die Richtung zeigt den nächsten Schritt in Richtung Ziel an und ist dargestellt als:

Code	Richtung
0	Rechts
1	Links
2	Oben
3	Unten

Tabelle 1: Codierung der Richtungen im Labyrinth

Der Kostenwert gibt an, wie viele Schritte bis zum Ziel benötigt werden. Ein Labyrinth mit dazugehöriger Matrix könnte so aussehen:

13;3	14;1	5;0	4;0	3;3
12;3	13;1	14;1	5;2	2;3
11;3	8;0	7;0	6;2	1;3
10;0	9;2	8;2	7;2	0>null

Abbildung 1: Labyrinth und Matrix

1.1 Erstellung der Matrix

Zu Beginn wird eine Matrix erstellt, deren Werte für alle Positionen zunächst leer sind. Ausgehend vom Zielpunkt durchläuft der Algorithmus rekursiv alle erreichbaren Nachbarpositionen. Für jeden Nachbarn wird in der Matrix ein Kostenwert eingetragen, der um eins höher ist als der des aktuellen Punktes, sowie die Richtung, aus der der Nachbar erreicht wurde.

1.2 Lösen der Labyrinth

Der Algorithmus funktioniert so, dass er jeweils den aktuell besten bekannten Weg auswählt diesen um einen weiteren Schritt erweitert. Den nächsten Schritt ermittelt er, indem er in der Matrix prüft, welchen Schritt die beiden Labyrinth für ihre jeweilige Position vorschlagen. Dieser Schritt wird am Ende der bisherigen Schrittfolge angehängt. Dabei entstehen in der Regel zwei unterschiedliche Wege. Beide werden zur Liste der bisherigen Wege hinzugefügt. Um den besten Weg auszuwählen, wird diese Rechnung angewandt:

$$\frac{c_b - c_a}{n} \quad (1)$$

wobei gilt:

- c_b sind die summierten Kosten von beiden Startpositionen

- c_a sind die summierten Kosten von beiden aktuellen Positionen

- n ist die Anzahl der benötigten Schritte

Der Bruch beschreibt also die durchschnittliche Einsparung pro Schritt – dabei ist 2 der maximal erreichbare Wert.

Der Weg mit der höchsten Einsparung gilt dabei als momentan bester Weg. Nach jeder Iteration wird das aktuelle Positionspaar $((x_1, y_1), (x_2, y_2))$ in einem Dictionary gespeichert, um zu verhindern, dass der

Algorithmus die exakt selben Positionen besucht. Zusätzlich gibt es auch die Begrenzung l , die festlegt, wie viele unterschiedliche Weg, sortiert nach ihren Kosten, in der Liste gespeichert werden dürfen. Diese Begrenzung beeinflusst maßgeblich die Laufzeit und Genauigkeit des Algorithmus. In der Liste dürfen die Wege unterschiedliche Werte für n haben; wichtig ist lediglich, dass ihre Kosten (Gleichung (1)) gering sind. Sobald ein Weg besucht wurde, wird er aus der Liste entfernt. Der gesamte Ablauf wiederholt sich solange, bis beide im Ziel sind.

1.3 Gruben

Gruben werden sowohl in der Kostenmatrix als auch beim Lösen wie normale Felder gesehen. Berührt jedoch einer der beiden eine Grube, wird seine Position auf die jeweilige Startposition zurückgesetzt. Dadurch steigen die Kosten des Wegs so weit an, dass dieser in der Regel aus der Liste der potenziellen Wege entfernt wird.

2 Umsetzung

Die Lösung wurde in Python3 geschrieben. Die Eingabetexte werden eingelesen. Für beide Labyrinth werden die horizontalen und vertikalen Wände jeweils in separaten Liste gespeichert. Auch die Positionen der Gruben werden für jedes Labyrinth in einer eigenen Liste erfasst.

2.1 Erstellung der Matrix

Zur Erstellung der Matrix wird die Klasse *cost* benutzt.

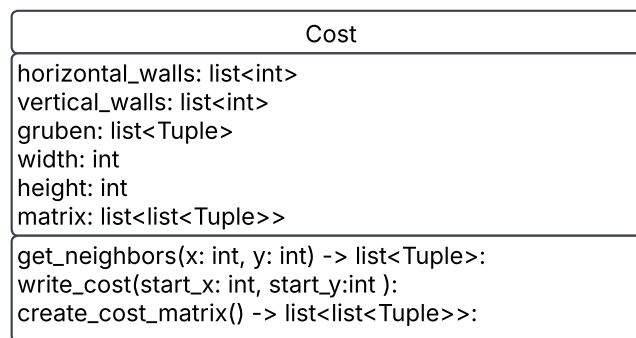


Abbildung 2: Cost Klasse

Für jedes Labyrinth wird ein Cost-Objekt erstellt und die Methode

```
1 cost_obj.create_cost_matrix()
```

aufgerufen. Das Cost Objekt bekommt als Übergabewerte die horizontalen und vertikalen Wände, die Gruben sowie die Größe des Labyrinths. Die *create_cost_matrix* Methode erstellt zunächst eine leere Liste mit der Dimension [height][width], die als Kostenmatrix dient. Dann ruft sie die Methode *write_cost* auf um die Kostenmatrix zu füllen.

```
1 def write_cost(self, start_x: int, start_y: int):
```

Listing 1: Methode *write_cost*

Zu Beginn wird die Liste *stack* initialisiert. Sie speichert alle Felder, die noch besucht werden müssen. Die Startposition wird als erstes Element hinzugefügt. Jeder Eintrag in der Liste besitzt vier Werte: der x-Position, der y-Position, den aktuell Kosten sowie der entgegengesetzten Richtung, aus der das Feld betreten wurde. Anschließend wird das erste Element der Liste entnommen und in die Kostenmatrix übertragen. Falls es schon in der Kostenmatrix ist, wird es übersprungen. Daraufhin werden alle benachbarten Felder mit der Methode *get_neighbours* ermittelt und dem Stack hinzugefügt. Dabei werden die Kosten um eins erhöht und die jeweilige Richtung gespeichert. Das wiederholt sich solange bis der Stack leer ist.

2.2 Lösen der Labyrinth

Zum lösen der Labyrinth wird die Klasse *Solving* benutzt.

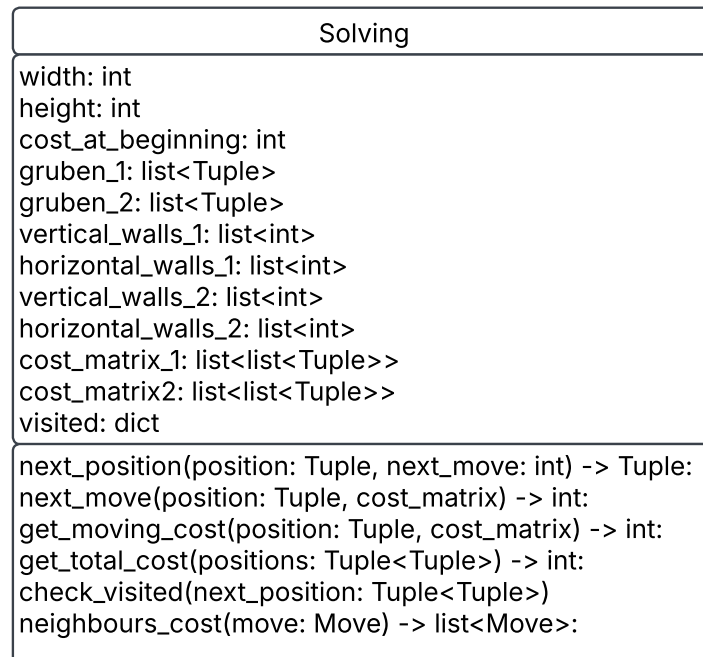


Abbildung 3: Solving Klasse

Die Wege werden mit der Klasse *Move* dargestellt.

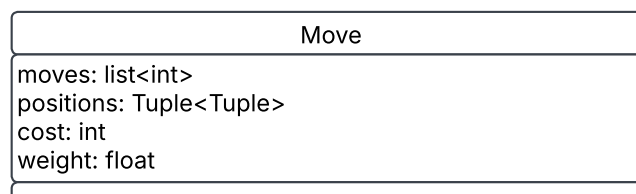


Abbildung 4: Moving Klasse

Das Attribut *cost* beschreibt die aufsummierten Kosten der beiden aktuellen Positionen basierend auf den Werten aus den Kostmatrizen. das Attribut *weight* wird gemäß der Gleichung (1) berechnet.

Zu Beginn wird ein *Solving*-Objekt initialisiert und eine leere Liste *moves* angelegt, in der alle aktuelle Wege gespeichert werden. Anschließend wird ein erstes *Move*-Objekt erzeugt, dessen Startposition ((0, 0), (0, 0)) ist und seine moves leer sind. In jedem Schritt wird das erste Element aus der *moves* Liste entnommen. Dann wird mit folgendem Aufruf

```
1 new_moves = solving_obj.neighbours_cost(move)
```

eine Liste von möglichen Folgewege erzeugt Die Liste wird mit den neun Wege erweitert, sortiert und auf die *l* besten Wege beschränkt.

```
1 def neighbours_cost(self, move: Move) -> list:
```

Listing 2: Methode *neighbours_cost*

Zu Beginn werden die beiden Positionen aus dem Attribut *positions* der übergebenen *move*-Objekt genommen. Anschließend werden mit den folgenden Aufrufen:

```
1  move_1 = self.next_move(pos1, self.cost_matrix_1)
   move_2 = self.next_move(pos2, self.cost_matrix_2)
```

die jeweils nächsten möglichen Richtungen aus der Kostenmatrix bestimmt. Für beide Züge werden daraufhin die neuen Positionen berechnet:

```
2  next_position_1 = self.next_position(move.positions, move_1)
   next_position_2 = self.next_position(move.positions, move_2)
```

Dabei ist *next_position* ein Positionspaar der Form $((x_1, y_1), (x_2, y_2))$. Mit dem folgenden Aufruf wird Anschließend überprüft, ob die neuen Positionen bereits besucht wurden:

```
2  move_1 = self.check_visited(next_position_1, len(move.moves) + 1, move_1)
   move_2 = self.check_visited(next_position_2, len(move.moves) + 1, move_2)
```

Zum Schluss wird ein neues *Move*-Objekt aus beiden möglichen Zügen erstellt und übergeben. Falls eine der neuen Positionen bereits besucht wurde oder bereits im Ziel ist, wird der entsprechende Zug nicht übergeben.

3 Laufzeit und Diskussion

Genügend Beispiele einbinden! Die Beispiele von der BwInf-Webseite sollten hier diskutiert werden, aber auch eigene Beispiele sind sehr gut – besonders wenn sie Spezialfälle abdecken. Aber bitte nicht 30 Seiten Programmausgabe hier einfügen!

4 Beispiele

Genügend Beispiele einbinden! Die Beispiele von der BwInf-Webseite sollten hier diskutiert werden, aber auch eigene Beispiele sind sehr gut – besonders wenn sie Spezialfälle abdecken. Aber bitte nicht 30 Seiten Programmausgabe hier einfügen!

5 Quellcode

Unwichtige Teile des Programms sollen hier nicht abgedruckt werden. Dieser Teil sollte nicht mehr als 2–3 Seiten umfassen, maximal 10.